

HTML Dashboard Guide

CRE Flood Graph Dashboard — Technical Structure & Usage

File: CRE_Flood_Graph_Dashboard_Standalone.html
Size: ~598 lines (single self-contained HTML file)
Dependencies: External CDN libraries (requires internet connection)

1. Architecture Overview

The dashboard is a **single standalone HTML file** that contains:

- All HTML structure
- All CSS styles (embedded in `<style>` block)
- All JavaScript logic (embedded in `<script>` block)
- **All data embedded as inline JSON** (the `EMBEDDED_DATA` constant)

External CDN Dependencies

Library	Version	Purpose
Leaflet.js	1.9.4	Interactive maps
Leaflet CSS	1.9.4	Map styling
Chart.js	4.4.0	All charts (bar, line, dough-nut)
chartjs-plugin-datalabels	2.2.0	Data labels on charts

 The dashboard requires an internet connection to load these CDN libraries. It will NOT work fully offline.

2. Three Dashboard Pages

Page 1: Executive Overview (#page1)

Purpose: High-level view of the entire CRE development pipeline across all cities and sectors.

Components:

Component	Type	Description
Slicers	Dropdowns + Inputs	Filter by City, Sector, Status, and Year Range
KPI Cards (7)	Summary cards	Total Projects, Total SF, Total Units, Active Cities, Active Sectors, Est. Value, Avg Project Size

Project Map	Leaflet map	Circle markers sized by SF, colored by sector. Click for popup details.
SF by Sector	Doughnut chart	Square footage distribution across sectors
Cumulative SF (Flood Effect)	Stacked area chart	Cumulative square footage over time by sector — the signature “flood graph”
Projects by Sector & Year	Stacked bar chart	Project count by sector for each delivery year

Interactivity:

- All slicers (City, Sector, Status, Year Range) filter Page 1 charts in real time
- “Reset All” button clears all filters
- Map markers are clickable with popup showing project details

Page 2: Multifamily Deep-Dive (#page2)

Purpose: Focused analysis of multifamily/residential projects specifically.

Components:

Component	Type	Description
MF KPI Cards (6)	Summary cards	MF Projects, Total Units, Total SF, Avg Unit Density, Completion Rate, Pipeline Units
MF Project Map	Leaflet map	Only multifamily projects shown
MF Projects by Year	Combo chart	Bar (project count) + Line (unit count) by year
MF Status Distribution	Doughnut chart	Completed vs Under Construction vs Pipeline
Top MF Projects	Data table	Top 25 multifamily projects sorted by unit count

Note: Page 2 uses the FULL dataset (not filtered by Page 1 slicers). It always shows all multifamily data.

Page 3: Time Series Analysis (#page3)

Purpose: Temporal trends and growth patterns across the entire dataset.

Components:

Component	Type	Description
Annual Project Count	Bar chart	Number of projects per delivery year
Annual SF Delivered	Bar chart	Total square footage per delivery year
Sectors Over Time	Stacked bar chart	SF breakdown by sector for each year
Cumulative Growth	Dual-axis line chart	Cumulative SF (left axis) and cumulative projects (right axis)
Year-over-Year Growth Rate	Bar chart	% change in SF delivered vs prior year (green=positive, red=negative)

Note: Page 3 also uses the FULL dataset (not filtered by Page 1 slicers).

3. Data Embedding — Where the Data Lives

Location in Code

The data is embedded as a **JSON array** stored in a JavaScript constant at approximately **line 594** of the HTML file:

```
const EMBEDDED_DATA = [  
  {"project_name":"17xM","city":"DC","ward":2,...},  
  {"project_name":"Reunion Square (Building 4)","city":"DC",...},  
  // ... all 999 records  
];
```

Data Schema (per record)

Field	Type	Description
project_name	string	Name of the project
city	string	"DC" or "Atlanta"
ward	number	Ward/district number
neighborhood	string	Neighborhood or address
developer	string	Developer name(s)
architect	string	Architect (mostly empty)
sector	string	Office, Multifamily, Retail, Hotel, Other, Industrial, Mixed-Use
status	string	Completed, Under Construction, Pipeline
sqft	number	Square footage
units	number/string	Residential units (empty string if N/A)
est_value_millions	number	Estimated value in dollars (NOT millions despite field name)
est_delivery	string	Estimated delivery quarter/year (e.g., "Q2 24", "2029")
report_year	number	Year of the source report
latitude	number	GPS latitude
longitude	number	GPS longitude
delivery_year	number/string	Numeric year or empty string
delivery_quarter	number/string	Quarter (1-4) or empty string
delivery_date	string	ISO date or empty string

Important Data Quirk

⚠ The field `est_value_millions` contains values in **actual dollars**, NOT millions. For example, a value of `216000000.0` means \$216 million. The dashboard's KPI calculation accounts for this by displaying `{{fmtM(totalValue*1e6)}}` — which effectively double-counts the millions. This is a known naming inconsistency in the data.

4. How to Update the Data

Method 1: Replace the Embedded JSON (Simplest)

1. Open `CRE_Flood_Graph_Dashboard_Standalone.html` in a text editor
2. Find the line starting with `const EMBEDDED_DATA = [`
3. Replace the entire JSON array with your new data
4. Ensure the JSON matches the schema above
5. Save the file

Generate new JSON from CSV:

```
import pandas as pd
import json

df = pd.read_csv('your_new_data.csv')
# Convert to list of dicts, handling NaN
records = df.where(df.notna(), '').to_dict('records')
json_str = json.dumps(records)
print(f"const EMBEDDED_DATA = {json_str};")
```

Method 2: Load from External CSV File

To make the dashboard load data from an external file instead of embedded JSON, replace the init call at the end of the `<script>` block:

```
// Replace: init(EMBEDDED_DATA);
// With:
fetch('path/to/combined_pipeline.csv')
  .then(r => r.text())
  .then(csv => {
    const lines = csv.split('\n');
    const headers = lines[0].split(',');
    const data = lines.slice(1).filter(l => l.trim()).map(line => {
      const vals = line.split(',');
      const obj = {};
      headers.forEach((h, i) => {
        const v = vals[i]?.trim() || '';
        obj[h] = isNaN(v) || v === '' ? v : parseFloat(v);
      });
      return obj;
    });
    init(data);
  });
```

⚠ Loading external files requires serving the HTML from a web server (HTTP). Opening the file directly via `file://` will block fetch requests due to browser CORS policies.

Method 3: Add New Cities

To add a new city's data:

1. Prepare the data in the same CSV schema
2. Ensure the `city` field contains the new city name
3. Add GPS coordinates (`latitude` , `longitude`) for map functionality
4. Merge with existing data and regenerate the EMBEDDED_DATA JSON
5. The dashboard will automatically populate the City slicer with the new city

5. Styling & Customization

Color Scheme

Defined in CSS `:root` variables (line 12):

```
:root {
  --primary: #003366; /* Navy blue – headers, titles */
  --accent: #2196F3; /* Blue – buttons, highlights */
  --bg: #F5F5F5; /* Light gray – page background */
  --card-bg: #FFFFFF; /* White – card backgrounds */
  --text: #333333; /* Dark gray – body text */
  --text-light: #666666; /* Medium gray – labels, subtitles */
  --border: #E0E0E0; /* Light border color */
}
```

Sector Colors (JavaScript, line 206)

```
const SECTOR_COLORS = {
  'Office': '#2196F3', // Blue
  'Multifamily': '#FF9800', // Orange
  'Retail': '#4CAF50', // Green
  'Hotel': '#9C27B0', // Purple
  'Other': '#F44336', // Red
  'Industrial': '#00BCD4', // Cyan
  'Mixed-Use': '#607D8B' // Blue-gray
};
```

Status Colors (JavaScript, line 216)

```
const STATUS_COLORS = {
  'Completed': '#4CAF50', // Green
  'Under Construction': '#FF9800', // Orange
  'Pipeline': '#2196F3' // Blue
};
```

6. Navigation System

The navigation uses a simple tab system (line 225-234):

- Clicking a nav tab adds `.active` class to the tab and corresponding page `<div>`

- All pages are hidden by default (`display:none`) except the active one
- Map resize is triggered on tab switch to prevent rendering issues

7. Key Functions Reference

Function	Purpose
<code>init(data)</code>	Bootstrap — populates slicers, inits maps, triggers first render
<code>getFiltered()</code>	Returns data filtered by current slicer values
<code>updateAll()</code>	Master refresh — updates all KPIs, maps, and charts
<code>updateKPIs(data)</code>	Renders the 7 KPI cards on Page 1
<code>updateMap(map, markers, data, filterSector)</code>	Plots circle markers on Leaflet map
<code>updateDonutChart(data)</code>	SF by sector doughnut
<code>updateCumulativeChart(data)</code>	The “flood effect” cumulative area chart
<code>updateStackedBarChart(data)</code>	Projects by sector/year stacked bar
<code>updateMFKPIs()</code>	Multifamily-specific KPI cards
<code>updateMFCharts()</code>	All Page 2 charts + table
<code>updateTSCharts()</code>	All Page 3 time series charts
<code>resetFilters()</code>	Clears all slicers and refreshes

8. Responsive Design

The dashboard uses CSS Grid with responsive breakpoints:

- **Desktop (>900px):** Multi-column layouts (2-col, 3-col, map+charts)
- **Mobile (<900px):** All grids collapse to single column; KPI cards become 2-column

9. Known Issues & Limitations

1. **Map tiles may fail** — The tile URL uses a static Wikipedia image as a fallback, not a proper tile server. For production, replace with OpenStreetMap or Mapbox tiles.
2. **No data persistence** — Filters reset on page reload; no state is saved.

3. **Page 2 & 3 ignore Page 1 filters** — This is by design but may confuse users.
4. **Large embedded data** — The ~999-record JSON makes the HTML file large (~300KB+). For datasets over 5,000 records, consider external loading.
5. **No error handling** — If data is malformed, charts will silently fail.